Abstract Class in Java

Overview

In Java, an **abstract class** is a class that is declared with the `abstract` keyword. It serves as a blueprint for other classes and cannot be instantiated directly. Abstract classes are primarily used to define a common interface and shared behavior for their subclasses.

Key Characteristics of Abstract Classes

1. **Cannot be instantiated:**
   - You cannot create an object of an abstract class.
   - Example:
   - ```
     abstract class Shape {
         abstract void draw();
     }

     // Compilation error:
     Shape shape = new Shape();
     ```
2. **May have abstract methods:**
   - Abstract methods are methods without a body (implementation).
   - Subclasses must provide implementations for all abstract methods unless they are also abstract.
3. **Can have concrete methods and fields:**
   - Unlike interfaces, abstract classes can include concrete methods (with implementation) and instance variables.
4. **Can have constructors:**
   - Constructors in abstract classes are called when an instance of a subclass is created.
5. **Can have static methods:**
   - Static methods in abstract classes can be used without inheriting the class.

Abstract Methods

An abstract method is declared using the `abstract` keyword and has no body. Abstract methods must be implemented by the first concrete subclass.

Example:

```
abstract class Animal {
    // Abstract method
    abstract void sound();
```
```

```
    // Concrete method
    void eat() {
        System.out.println("This animal eats food.");
    }
}

class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog says Woof!");
    }
}

public class Main {
    public static void main(String[] args) {
        Animal dog = new Dog();
        dog.sound(); // Output: Dog says Woof!
        dog.eat();   // Output: This animal eats food.
    }
}
```

Role of Constructors in Abstract Classes

Even though abstract classes cannot be instantiated, they can have constructors. These constructors are called when a subclass is instantiated. The primary purpose of constructors in abstract classes is to initialize fields and perform setup tasks common to all subclasses.

Example:

```
abstract class Vehicle {
    String brand;

    // Constructor
    Vehicle(String brand) {
        this.brand = brand;
        System.out.println("Vehicle constructor called");
    }

    abstract void drive();
}

class Car extends Vehicle {
    Car(String brand) {
        super(brand); // Calls the constructor of the abstract class
    }

    @Override
    void drive() {
        System.out.println(brand + " car is driving.");
    }
}

public class Main {
```

```
    public static void main(String[] args) {
        Vehicle car = new Car("Tesla");
        car.drive();
    }
}
```

Output:

```
Vehicle constructor called
Tesla car is driving.
```

Abstract Classes vs Interfaces

Feature	Abstract Class	Interface
Instantiation	Cannot be instantiated	Cannot be instantiated
Abstract Methods	Can have abstract methods	All methods are abstract (until Java 8)
Concrete Methods	Can have concrete methods	Can have default and static methods (Java 8+)
Fields	Can have fields (with or without `final`)	Can only have `public static final` fields
Constructors	Can have constructors	Cannot have constructors
Multiple Inheritance	Single inheritance allowed	Supports multiple inheritance (via multiple interfaces)

When to Use Abstract Classes

- When you want to provide common functionality to subclasses while still enforcing a contract (through abstract methods).
- When the class represents a base concept that logically should not be instantiated on its own (e.g., `Animal`, `Shape`, `Vehicle`).

Programming Examples

Abstract Class with Abstract and Concrete Methods

```
abstract class Shape {
    abstract void draw();

    void display() {
        System.out.println("This is a shape.");
```

```
    }
}

class Circle extends Shape {
    @Override
    void draw() {
        System.out.println("Drawing a circle.");
    }
}

public class Main {
    public static void main(String[] args) {
        Shape shape = new Circle();
        shape.draw();    // Output: Drawing a circle.
        shape.display(); // Output: This is a shape.
    }
}
```

Abstract Class for Common Properties

```
abstract class Employee {
    String name;
    int id;

    Employee(String name, int id) {
        this.name = name;
        this.id = id;
    }

    abstract void work();
}

class Developer extends Employee {
    Developer(String name, int id) {
        super(name, id);
    }

    @Override
    void work() {
        System.out.println(name + " is writing code.");
    }
}

public class Main {
    public static void main(String[] args) {
        Employee emp = new Developer("Alice", 101);
        emp.work(); // Output: Alice is writing code.
    }
}
```

Common Interview Questions

1. **What is an abstract class in Java? Why can't we instantiate it?**

- Abstract classes are incomplete classes designed to be extended. They cannot be instantiated because they may contain abstract methods without implementation.

2. **Can an abstract class have a constructor? If so, why?**
   - Yes, abstract classes can have constructors. These constructors are invoked during the instantiation of subclasses to initialize fields and perform setup tasks.
3. **What is the difference between an abstract class and an interface?**
   - Refer to the table in the section "Abstract Classes vs Interfaces."
4. **Can an abstract class have static methods?**
   - Yes, abstract classes can have static methods. These methods belong to the class and are not tied to any instance.
5. **What happens if a subclass does not implement all abstract methods of the abstract class?**
   - The subclass must also be declared abstract. Otherwise, a compilation error occurs.
6. **Can an abstract class implement an interface?**
   - Yes, an abstract class can implement an interface. However, it must provide implementations for the interface's methods or remain abstract.
7. **Can we declare an abstract class as `final`? Why or why not?**
   - No, an abstract class cannot be declared `final` because `final` classes cannot be extended, and abstract classes are meant to be extended.